

CSE721 Project Requirements

Frontend to Backend Plan

Implementation and Security Analysis of Classical and Modern Cryptographic Algorithms

Prepared as a simple implementation guide based on the CSE721 project requirements. Database/storage has been intentionally removed from this plan.

Area	Simple Meaning
Frontend	The user interface where users choose an algorithm, enter inputs, click buttons, and see results.
Backend	The program logic that implements each cryptographic algorithm from scratch and returns results.
No Database	The system does not need to save history, users, or previous results. Each request can be processed live.
Analysis	The system should show outputs such as ciphertext, decrypted text, round keys, attack results, timing, and comparison.

1. Project Overview

Goal: Build a simple website, app, or command-line system where users can select a cryptographic algorithm, provide the required input, and see encryption, decryption, key-generation, attack, and performance-analysis results.

Category	Algorithms Required	Main Output
Classical cryptography	Substitution cipher, Double transposition	Ciphertext, decrypted plaintext, frequency analysis
Symmetric-key cryptography	DES, AES	Ciphertext, decrypted plaintext, auto-generated key, all round keys
Public-key cryptography	RSA, ECC	Keys, ciphertext, decrypted message, factorization attack, shared key

Important implementation rule: do not use built-in cryptographic libraries for the core algorithm operations. Implement the core logic manually. Basic math and big-integer support are acceptable.

2. Recommended System Architecture Without Database

Simple architecture: Frontend sends input to backend. Backend runs the selected algorithm and immediately sends the output back. No data is stored permanently.

Layer	Responsibility	Example Technology
Frontend	Algorithm selection, input forms, buttons, result display, comparison table	HTML/CSS/JavaScript or React
Backend API	Receives inputs, calls correct algorithm module, returns JSON result	Python Flask or FastAPI
Algorithm modules	Manual implementation of substitution, transposition, DES, AES, RSA, ECC	Python files/modules
Utility modules	Input cleaning, timing, frequency analysis, formatting output	Python helper functions

System flow:

1. User selects an algorithm from the frontend.
2. Frontend shows the correct input form for that algorithm.
3. User enters plaintext, key, key size, or curve parameters depending on the algorithm.
4. Frontend sends the input to a backend API endpoint.
5. Backend validates input and calls the relevant algorithm module.
6. Backend calculates encryption, decryption, keys, attack result, timing, or frequency analysis.
7. Backend returns a JSON response.
8. Frontend displays the result clearly in output cards or tables.

3. Frontend Requirements

The frontend should be simple and user-friendly. It does not need login, signup, file upload, or database history.

3.1 Main Frontend Pages or Sections

Page/Section	What It Should Show
Home / Dashboard	Project title, short description, and six algorithm cards/buttons.
Classical Crypto	Substitution cipher and double transposition forms.
Symmetric Crypto	DES and AES forms with auto-generated keys and round-key outputs.

Public-Key Crypto	RSA and ECC forms for key generation, encryption/decryption, and ECDH.
Comparison / Analysis	Table comparing speed, security level, key type, attack type, and observations.

3.2 Frontend Components

- Navigation bar: Home, Classical, Symmetric, Public-Key, Comparison.
- Algorithm cards: one card/button for each algorithm.
- Input form: fields change depending on selected algorithm.
- Action buttons: Encrypt, Decrypt, Generate Keys, Analyze, Run Attack, Run ECDH.
- Result panel: shows ciphertext, plaintext, keys, round keys, valid points, or shared key.
- Error message area: shows clear messages for invalid input.
- Loading state: optional simple message like "Processing..." while backend runs.

4. Frontend Forms by Algorithm

Algorithm	Frontend Inputs	Frontend Buttons	Frontend Outputs
Substitution cipher	Plaintext; 26-letter permutation key	Encrypt; Decrypt; Frequency Analysis	Ciphertext; original plaintext; letter-frequency result
Double transposition	Plaintext; first permutation key; second permutation key	Encrypt; Decrypt; Frequency Analysis	Permuted ciphertext; original plaintext; frequency analysis
DES	Plaintext only; key auto-generated by backend	Encrypt/Decrypt	Ciphertext; decrypted plaintext; generated key; all round keys
AES	Plaintext only; key auto-generated by backend	Encrypt/Decrypt	Ciphertext; decrypted plaintext; generated key; all round keys
RSA	Choice: generate keys/encrypt/decrypt; plaintext; key size	Generate Keys; Encrypt; Decrypt; Factorization Attack	Public key; private key; ciphertext; decrypted message; optional attack result
ECC	p, a, b, G, n ; primitive point $P(x,y)$; private values a and b	Generate Points; Generate Keys; Run ECDH	Valid points; private key; public key; shared key

5. Backend Requirements

The backend is responsible for all cryptographic logic. The frontend should not perform the core encryption or decryption logic. It should only collect input and display output.

5.1 Suggested Backend Folder Structure

```

backend/
  app.py
  classical/
    substitution.py
    double_transposition.py
  symmetric/
    des.py
    aes.py
  public_key/
    rsa.py
    ecc.py
  utils/
    frequency_analysis.py

```

timing.py
validators.py
formatters.py

5.2 Backend Module Responsibilities

Module	Main Functions
substitution.py	Validate 26-letter key; encrypt text; decrypt text; map plaintext letters to cipher letters.
double_transposition.py	Apply first permutation; apply second permutation; reverse both permutations for decryption.
des.py	Generate key; create 16 round keys; implement initial permutation, Feistel rounds, S-boxes, and final permutation.
aes.py	Generate key; expand round keys; implement SubBytes, ShiftRows, MixColumns, AddRoundKey, and inverse operations.
rsa.py	Generate primes; calculate n, phi, e, d; encrypt; decrypt; optional factorization attack for small key sizes.
ecc.py	Validate curve; list points; point addition; scalar multiplication; public-key generation; ECDH shared key.
frequency_analysis.py	Count letters; calculate percentage; compare frequency distribution for classical ciphers.
validators.py	Check empty plaintext, invalid key length, wrong permutation, invalid curve parameters, etc.
timing.py	Measure encryption/decryption/key-generation time for comparison.

6. Suggested Backend API Endpoints

Endpoint	Method	Purpose
/api/substitution/encrypt	POST	Encrypt plaintext using a 26-letter substitution key.
/api/substitution/decrypt	POST	Decrypt ciphertext using the same substitution key.
/api/substitution/frequency	POST	Return letter-frequency analysis.
/api/double-transposition/encrypt	POST	Encrypt plaintext using two permutation keys.
/api/double-transposition/decrypt	POST	Decrypt ciphertext by reversing both transpositions.
/api/des/run	POST	Generate key, encrypt, decrypt, and return DES round keys.
/api/aes/run	POST	Generate key, encrypt, decrypt, and return AES round keys.
/api/rsa/generate-keys	POST	Generate RSA public and private keys based on key size.
/api/rsa/encrypt	POST	Encrypt plaintext using RSA public key.
/api/rsa/decrypt	POST	Decrypt ciphertext using RSA private key.
/api/rsa/factorization-attack	POST	Try factorization attack for demonstration, preferably on small keys only.
/api/ecc/points	POST	List valid points on the elliptic curve.

/api/ecc/generate-key	POST	Generate ECC public key from private key and base point.
/api/ecc/ecdh	POST	Calculate shared key for Elliptic Curve Diffie-Hellman.

7. Input Validation Rules

Algorithm	Validation Needed
Substitution cipher	Key must contain exactly 26 unique alphabetic letters. Plaintext should not be empty.
Double transposition	Permutation keys must contain valid positions with no duplicate index. Plaintext can be padded if needed.
DES	Plaintext should be converted to block format. Key must be generated in correct DES size.
AES	Plaintext should be padded to block size. Key size should match selected AES version, e.g., 128-bit.
RSA	Key size should be valid; generated p and q must be prime; plaintext must be converted to integer safely.
ECC	Curve must satisfy valid elliptic curve conditions; points must lie on the curve; private keys must be in range.

8. Detailed Algorithm Requirements

8.1 Substitution Cipher

- Frontend takes plaintext and a 26-letter permutation key.
- Backend creates a mapping from A-Z to the key letters.
- Encryption replaces each plaintext letter with the mapped cipher letter.
- Decryption uses the reverse mapping.
- Frequency analysis counts how many times each letter appears in the ciphertext.
- Output should include ciphertext, decrypted plaintext, and frequency table or chart.

8.2 Double Transposition

- Frontend takes plaintext, first permutation key, and second permutation key.
- Backend arranges plaintext into blocks or a grid based on key size.
- First permutation reorders columns or positions using the first key.
- Second permutation applies another reordering using the second key.
- Decryption reverses the second permutation first, then reverses the first permutation.
- Output should include encrypted ciphertext, decrypted plaintext, and frequency-analysis result.

8.3 DES

- Frontend only needs plaintext input because key is auto-generated.
- Backend generates a DES key and creates all round keys.
- Implement core DES manually: initial permutation, expansion, XOR, S-box substitution, P-box permutation, 16 Feistel rounds, and final permutation.
- Output should include ciphertext, decrypted plaintext, generated key, and all round keys.
- Also measure encryption and decryption time for comparison.

8.4 AES

- Frontend only needs plaintext input because key is auto-generated.

- Backend generates AES key and performs key expansion.
- Implement core AES manually: SubBytes, ShiftRows, MixColumns, AddRoundKey, and inverse operations for decryption.
- Return all round keys so the process is visible to the user.
- Output should include ciphertext, decrypted plaintext, generated key, and round-key list.

8.5 RSA

- Frontend provides choice: generate keys, encrypt, or decrypt.
- For key generation, user enters key size such as 512 or 1024 bits.
- Backend generates primes p and q , computes n , $\phi(n)$, public exponent e , and private exponent d .
- Encryption converts plaintext into integer or hex representation and calculates ciphertext.
- Decryption converts ciphertext back to original message.
- Optional factorization attack should be shown only for small/demo key sizes because large RSA keys are not practical to factor in a class project.

8.6 ECC

- Frontend takes curve parameters p , a , b , base point G , order n , and primitive point $P(x, y)$.
- Backend verifies that the curve and points are valid.
- Backend lists valid points on the curve for small demonstration parameters.
- Private key is a random or user-provided integer in the valid range.
- Public key is calculated using scalar multiplication: private key times base point.
- For ECDH, both users generate public keys and backend calculates the shared key.

9. Output Design

Output Type	How to Display It
Ciphertext	Show in a boxed text area. Allow copy button if using a website.
Decrypted plaintext	Show below ciphertext to prove correctness.
Keys	Show public/private/generated keys clearly. For long keys, use collapsible section.
Round keys	Show in a table: round number and key value.
Frequency analysis	Show letter counts and percentages; optional bar chart.
Attack result	Show whether attack succeeded, time taken, and explanation.
Performance	Show encryption time, decryption time, and key-generation time.

10. Security and Performance Analysis

The project should not only show working encryption and decryption. It should also compare algorithms in simple terms.

Algorithm	Key Type	Security Note	Analysis to Show
Substitution	Classical fixed mapping	Weak because letter patterns remain visible.	Frequency analysis.
Double transposition	Classical permutation keys	Stronger than simple substitution but still weak by modern standards.	Frequency and structure observation.
DES	Symmetric key	Historically important but weak today due to small key size.	Round keys and timing.

AES	Symmetric key	Modern and strong when implemented correctly.	Round keys and timing.
RSA	Public/private key	Security depends on difficulty of factoring n .	Key size, encryption/decryption time, factorization demo.
ECC	Public/private key	Strong security with smaller keys compared with RSA.	Point generation, public key, shared key.

11. Implementation Plan

9. Create frontend layout with six algorithm cards.
10. Create backend project structure and one health-check API.
11. Implement Substitution Cipher because it is the easiest starting point.
12. Add frequency-analysis utility and connect it to classical ciphers.
13. Implement Double Transposition encryption and decryption.
14. Implement RSA key generation, encryption, decryption, and small-key factorization demo.
15. Implement ECC point operations and ECDH demonstration with small parameters first.
16. Implement DES manually and display all round keys.
17. Implement AES manually and display all round keys.
18. Add performance timing for every algorithm.
19. Create final comparison table and prepare demo screenshots.

12. Final Deliverables Checklist

- Working frontend with algorithm selection.
- Working backend with separate modules for each algorithm.
- No database or permanent storage required.
- Manual implementation of core cryptographic operations.
- Substitution cipher with encryption, decryption, and frequency analysis.
- Double transposition with encryption, decryption, and frequency analysis.
- DES with auto-generated key and all round keys.
- AES with auto-generated key and all round keys.
- RSA with key generation, encryption, decryption, and optional factorization attack.
- ECC with valid points, private key, public key, and ECDH shared key.
- Security and performance comparison table.
- Clean documentation and commented code.

13. Very Simple Final Summary

In simple words: The frontend is only for user interaction. The backend does all cryptographic calculations. The project does not need a database. The main requirement is to manually implement the algorithms, show encryption/decryption results, show keys or round keys where required, and compare the security and performance of all algorithms.